

# Laporan PBO Tugas 2

5025241134 Gilbran Mahavikia Raja

September 9, 2025

## 1 Russian Year 5 Problem

### MGSINFOB - Russian Year 5 Problem

*no tags*

Ms. G was looking at her collection of quality Russian year 5 problems when she noticed a pattern: there are always two answers that added up to 100. She assigned this problem to every Maths class at school: given a list of numbers (the answers to the problems), find the index of the two numbers that add up to 100. There will always be a unique solution.

#### Input

The input contains two lines.

The first line has an integer  $N$  ( $1 \leq N \leq 10^4$ ), the number of answers Ms. G provided you with.

The second line contains an array  $T$  of  $N$  integers ( $1 \leq T_i \leq 169$ ), the answers to the problems.

#### Output

Output two integers on two separate lines in order, the two index where the answers sum up to 100.

#### Example

```
Input:
3
40 60 29

Output:
1
2
```

#### Explanation

$40 + 60 = 100$ , 40 is at index 1, 60 is at index 2

Dalam soal diberikan sebuah daftar  $T$  berisi  $N$  angka ( $1 \leq N \leq 10^4$ ), setiap  $T[i]$  berada dalam rentang 1 hingga 169, dan diketahui bahwa terdapat tepat dua angka dalam daftar yang jika dijumlahkan menghasilkan 100.

## 2 Identifikasi Masalah

Masalahnya adalah mencari dua unsur dalam array yang jumlahnya sama dengan target tertentu (100). Domain klasik ini disebut “two-sum”, dengan kondisi:

- $N \leq 10^4$ , sehingga solusi  $O(N^2)$  masih mungkin tetapi mungkin terancam waktu tergantung konstanta, sedangkan  $O(N)$  sangat ideal.
- Nilai-nilai kecil (maks 169), memperbolehkan juga pendekatan counting atau lookup tabel kecil.

- Dinyatakan solusi unik, jadi hanya satu pasangan yang tepat.
- Dalam Komentar di soal ada yang mengeluhkan solusi  $O(N^2)$  menghasilkan WA sementara versi berbasis hashmap AC.

### 3 Solusi dan implementasi pada C++

Solusi masalah MGSINFOB bisa diselesaikan dengan memanfaatkan *unordered\_map* di C++ untuk melakukan pencarian pasangan bilangan secara cepat. Caranya, kita buat sebuah *unordered\_map*  $\langle \text{int}, \text{int} \rangle$  yang berfungsi menyimpan nilai dari elemen array sebagai key dan indeksnya (1-based) sebagai value. Kemudian, saat kita melakukan iterasi pada array, untuk setiap elemen  $x = T[i]$  kita hitung nilai pasangan yang dibutuhkan, yaitu  $y = 100 - x$ . Jika  $y$  sudah ada di dalam *unordered\_map*, berarti kita telah menemukan pasangan angka yang jumlahnya 100, maka langsung cetak indeks  $y$  (yang sudah tersimpan di map) dan indeks  $x$  saat ini ( $i + 1$ ). Karena *unordered\_map* mendukung operasi pencarian (find) dalam waktu konstan rata-rata  $O(1)$ , seluruh algoritma berjalan dalam kompleksitas  $O(N)$ . Setelah mencetak kedua indeks tersebut, program bisa langsung berhenti karena soal menjamin solusi unik. Dengan pendekatan ini, kita tidak perlu lagi menggunakan nested loop seperti pada brute-force, sehingga lebih efisien dan aman dari kesalahan urutan indeks.

```

1  #include <iostream>
2  #include <unordered_map>
3  using namespace std;
4
5  int main()
6  {
7      int t, index = 1;
8      bool found = false;
9      unordered_map<int, int> umap;
10     scanf("%d", &t);
11     while (t--)
12     {
13         int temp;
14         scanf("%d", &temp);
15         if (umap.find(100-temp) == umap.end())
16         {
17             umap[temp] = index;
18         }
19         else
20         {
21             printf("%d\n%d\n", umap[100-temp], index);
22             found = true;
23         }
24         index++;
25     }
26     return 0;
27 }
```

### 4 Bukti

Berikut adalah bukti bahwa solusi saya mendapatkan verdict **Accepted**.

|          |                        |              |                        |                            |      |      |       |
|----------|------------------------|--------------|------------------------|----------------------------|------|------|-------|
| 34959575 | 2025-09-08<br>14:52:42 | GilbranRP134 | Russian Year 5 Problem | accepted<br>100% (100/100) | 0.01 | 5.2M | CPP14 |
|----------|------------------------|--------------|------------------------|----------------------------|------|------|-------|